Tarea Programada

Modelo Conjunto (Unión, Intersección y Diferencia)

Segunda Etapa

Análisis de Algoritmos y Estructuras de Datos

Docente: Braulio Jose Solano Rojas

Sebastian Arroyo Marin C5l364
Julián F. Vargas Redondo C4K773

Lenguaje utilizado: Python

Tabla de contenidos

Modelo Conjunto **2**
Lista Ordenada Genérica **2**
Propiedades 3
Operaciones 3
Lista Ordenada por punteros simplemente enlazada **3**
Propiedades 4
Operaciones 4
Lista Ordenada por arreglos **5**
Propiedades 5
Operaciones 5
Tabla Hash Genérica: **6**
Propiedades 6
Operaciones 6
Tabla Hash Abierta **7**
Propiedades 7
Operaciones 7
Función Hash 8
Redistribución de su tiempo de duración 9
Árbol de Búsqueda Binaria (ABB) genérico **9**
Propiedades 9
Operaciones del Modelo Conjunto 10
ABB por punteros **10**
Propiedades y funcionamiento 10
A.B.B. por vector heap **10**
Propiedades y funcionamiento 10
E.D Genérica Trie **11**
Descripción 11
E.D. Trie por punteros **12**
Descripción 12
Operaciones 12
E.D. Trie por arreglos **13**
Propiedades y funcionamiento 13

Modelo Conjunto

El Modelo Conjunto representa una colección de elementos sin repetición y sin un orden inherente para el usuario. Este modelo cuenta con ciertas propiedades que son:

- Los elementos son únicos: un elemento no puede aparecer más de una vez dentro de un mismo conjunto.
- El conjunto no tiene posiciones ni índices, lo único que importa es si pertenece o no al conjunto.
- Los elementos serán alfanuméricos (tipo string) con una longitud máxima de 20 caracteres. Los elementos podrán ser letras en el rango de a-z y números de 0-9.

El modelo Conjunto debe de tener las siguientes operaciones:

- **Init():** init inicializa el conjunto vacío
- **Done():** libera toda la memoria utilizada por el conjunto. Esto finaliza la utilización de la estructura de datos
- **Clear():** elimina todos los elementos de un conjunto, dejando el conjunto vacío.
- **Insert(elemento):** inserta el elemento en un conjunto. No se requiere que no exista, en el caso de que ya pertenezca al conjunto la operación no tiene efecto.
- **Delete(elemento):** elimina el elemento del conjunto si pertenece a ese conjunto.
- **Member(elemento):** Indica si el elemento pertenece o no al conjunto.
- **Print():** imprime los elementos del conjunto
- **Union(A, B):** crea un nuevo conjunto con los elementos que pertenecen a A y B, sin repetidos.
- **Intersection(A, B):** crea un nuevo conjunto con los elementos que pertenecen a ambos conjuntos.
- **Difference(A, B):** crea un nuevo conjunto con los elementos que pertenecen a A, pero no a B.

El Modelo Conjunto no establece ningún orden de los elementos. Sin embargo, algunas estructuras podrían mantenerlos ordenados internamente.

La implementación debe de mantener en todo momento la propiedad que dice que cada elemento es único en un mismo conjunto, por este motivo es que la operación **Insert** sobre un elemento que ya pertenece al conjunto no debe generar ningún efecto en la estructura.

Dentro de la implementación se pueden aprovechar las características de la estructura de datos para facilitar las operaciones. Por ejemplo, realizar recorridos con una Lista Ordenada para realizar Union, Intersection y Difference.

Lista Ordenada Genérica

Una Lista Ordenada es una estructura de datos lineal en la cual se mantiene el orden según el criterio. En este proyecto se utiliza una lista ordenada para los elementos alfanuméricos y estos se ordenan de manera ascendente.

Dentro del Modelo Conjunto una de sus propiedades menciona que la estructura no tiene posiciones ni índices, pero para el caso de la lista es muy útil mantenerla ordenada como propiedad interna, ya que se pueden realizar algunas operaciones de manera más eficiente.

Hasta este punto se explica la Lista Ordenada Genérica, sin embargo, esta estructura se puede implementar utilizando diferentes representaciones. Se puede implementar como una lista simplemente enlazada mediante punteros o un arreglo. Ambas implementaciones mantienen el comportamiento lógico de la Lista Ordenada, pero su forma de almacenar y modificar los elementos es diferente.

Propiedades

- Los elementos se mantienen ordenados de forma ascendente.
- Los elementos se comparan para determinar su posición dentro de la estructura.
- La estructura puede estar vacía.
- La inserción de un elemento debe realizarse en la posición que permita conservar el orden.
- Las operaciones de búsqueda pueden aprovechar el orden de los elementos para determinar anticipadamente que un elemento no se encuentra en la estructura.
- La representación interna puede variar sin cambiar el comportamiento de la estructura.

Operaciones

- **Init():** inicializa la lista vacía.
- **Done():** libera los recursos utilizados por la lista.
- **Clear():** elimina todos los elementos de la lista.
- **Insert(elemento):** busca la posición correspondiente e inserta el elemento, si el elemento ya existe, la operación no tiene ningún efecto.
- **Delete(elemento):** elimina el elemento dentro de la lista, si no existe, la operación no tiene ningún efecto.
- **Member(elemento):** determina si el elemento pertenece a la lista.
- **Print():** recorre e imprime los elementos almacenados en la lista.

- **Union(A, B):** crea una nueva lista ordenada que contiene los elementos de A y B sin duplicados.
- **Intersection(A, B):** crea una nueva lista ordenada con los elementos que pertenecen a ambos conjuntos.
- **Difference(A, B):** crea una nueva lista ordenada con los elementos que pertenecen a A pero no a B.

Lista Ordenada por punteros simplemente enlazada

Una lista enlazada es una estructura lineal que guarda una colección de elementos denominados nodos, donde cada nodo puede contener datos y un puntero al siguiente nodo. En el código se observa una Lista Ordenada mediante punteros, que en este caso es simplemente enlazada, ya que contamos con un nodo que señala al primer elemento de la lista; esto facilita que los nodos puedan ser ubicados en cualquier parte de la memoria, utilizando la referencia del nodo inicial dentro de la estructura.

En el Modelo Conjunto, la lista enlazada permite guardar los elementos y asegurar que no haya elementos duplicados. Para realizar una inserción, se recorre la lista hasta encontrar el punto exacto donde actual.elemento > elemento. Durante este recorrido se comprueba si el elemento ya existe, si existe, la operación no tendrá ningún efecto.

Propiedades

- Está conformada de nodos, donde cada nodo contiene un elemento y puntero hacia el siguiente nodo (en caso de que no haya siguiente, apunta a null).
- El recorrido de la lista solo puede realizarse en un único sentido.
- La lista guarda únicamente la referencia al primer nodo (cabeza o primero).
- La memoria se reserva conforme se insertan nuevos elementos, y se libera al eliminarlos o destruyendo la lista.
- Los nodos no requieren estar almacenados en posiciones seguidas de memoria.
- En cada nodo además de almacenar el elemento se requiere espacio adicional para almacenar el puntero.

Operaciones

- **Init():** init inicializa el conjunto vacío
- **Done():** libera los recursos utilizados por la estructura.
- **Clear():** elimina todos los elementos de la lista.
- **Insert(elemento):** Si la lista está vacía, el elemento se convierte en el primer nodo. Si el elemento es menor que el primer nodo, se inserta al inicio. En los demás casos, se recorre la lista hasta que actual sea “None” o actual.elemento sea >= elemento, para encontrar su posición correspondiente. Si el elemento ya existe, la operación no tiene ningún efecto.
- **Delete(elemento):** recorre la lista mientras actual no sea “Null” y actual.elemento sea menor a elemento. Si actual es “Null” o actual.elemento es diferente al elemento no

hace nada. Si anterior es “Null” el primer nodo pasa a ser el siguiente del actual, si no, el anterior.siguiente va a ser igual a actual.siguiente.

- **Member(elemento):** recorre la lista, si actual.elemento es igual al elemento retorna True, en caso contrario retorna False..
- **Print():** recorre e imprime los elementos de la lista en el orden en que se encuentran almacenados, finaliza mientras que actual no sea “Null”.
- **Union(A, B):** crea una nueva lista ordenada llamada “resultado ”que va a contener los elementos de A y B. Recorre primero A y agrega todos sus elementos a resultado, luego recorre B y agrega todos sus elementos a resultado.
- **Intersection(A, B):** crea una nueva lista ordenada llamada “resultado” que va a contener los elementos que pertenecen a ambos. Se recorre A y pregunta si el elemento actual es miembro de B, si es miembro, lo inserta a resultado, si no, continúa con el siguiente.
- **Difference(A, B):** crea una nueva lista ordenada llamada “resultado” con los elementos que pertenecen a A pero no a B. Se recorre A y pregunta si el elemento actual no pertenece a B, si no pertenece, lo agrega a resultado, si pertenece, continúa con el siguiente.

Lista Ordenada por arreglos

Una Lista Ordenada por arreglos es una estructura de datos lineal que almacena sus elementos ordenados por índices para identificar su posición.

Dentro del Modelo Conjunto, esta estructura permite almacenar los elementos de manera ordenada, evitando que un mismo elemento se repita. Mantener los elementos ordenados internamente facilita las operaciones de Member, Insert y Delete, así como las operaciones de Union, Intersection y Difference. La implementación que se utiliza en el proyecto se realiza mediante un recorrido secuencial del arreglo.

Propiedades

- Los elementos se almacenan en un arreglo y se mantienen ordenados de forma ascendente.
- Cada elemento ocupa una posición con un índice dentro del arreglo.
- Para acceder a una posición específica del arreglo se necesita un índice.
- Insert recorre la lista en busca de la posición donde colocar el elemento, mueve los elementos y lo coloca en la posición.
- El eliminar un elemento requiere de ordenar la lista para mantener el orden del arreglo.
- Member aprovecha el orden del arreglo para determinar si el elemento pertenece o no a la lista.
- La memoria utilizada es de O(n), donde n es el tamaño del arreglo.
- En el Modelo Conjunto, los elementos deben ser únicos, por lo que Insert no debe modificar la estructura si hay un elemento existente.

Operaciones

- **Init():** inicializa la lista vacía
- **Done():** libera los recursos utilizados por la lista.

- **Clear():** elimina todos los elementos de la lista.
- **Insert(elemento):** si la lista es de tamaño 0 ingresa el elemento en la posición 0. Mientras i < al tamaño del arreglo y elementos[i] < a elemento va recorriendo con iteraciones, al parar verifica si existe el elemento dentro del arreglo, si no existe lo agrega en la posición i, si existe, no tiene ningún efecto.
- **Delete(elemento):** utiliza la función member para saber si el elemento pertenece al arreglo, si sí existe, lo elimina del arreglo, si no, no tiene ningún efecto.
- **Member(elemento):** recorre el arreglo buscando si actual es igual al elemento, si se cumple la condición, retorna True, si no, retorna False.
- **Print():** recorre e imprime los elementos de la lista en el orden en que se encuentran almacenados, finaliza al acabar de imprimir todos los elementos.
- **Union(A, B):** crea un arreglo llamado “resultado” que va a contener los elementos de A y B. Recorre primero A y agrega todos sus elementos a resultado, luego recorre B y agrega todos sus elementos a resultado, retorna el arreglo resultado.
- **Intersection(A, B):** crea un arreglo llamado “resultado” que va a contener los elementos que pertenecen a ambos conjuntos. Se recorre A y pregunta si el elemento actual es miembro de B, si es miembro, lo inserta a resultado, si no, pasa al siguiente elemento.
- **Difference(A, B):** crea un arreglo llamado “resultado” con los elementos que pertenecen a A pero no a B. Se recorre A y pregunta si el elemento actual no pertenece a B, si no pertenece, lo agrega a resultado, si pertenece, pasa al siguiente elemento.

Tabla Hash Genérica:

Las tablas hash son estructuras de datos que se utilizan para almacenar un número elevado de datos sobre los que se necesitan operaciones de búsqueda e inserción muy eficientes. Una tabla hash almacena un conjunto de pares “(posición, valor)”. La posición es única para cada elemento de la tabla y es el dato que se utiliza para buscar un determinado valor.

Dentro del Modelo Conjunto, esta estructura permite almacenar los elementos de manera que las operaciones sean O(1), dando acceso de forma muy eficiente para cada conjunto y las operaciones entre ellos de Union, Intersection y Difference.

Propiedades

- Se accede directamente a los elementos a través de la posición que fue creada por la función hash.
- Los elementos no están ordenados. Su posición se define a través del resultado de la función hash.
- Existe la posibilidad que dos elementos diferentes generen la misma posición, debido a esto, toda tabla hash requiere de una función que evite este tipo de colisiones.
- Debido al funcionamiento de la tabla hash con posiciones únicas para almacenar sus elementos, las operaciones de Insert, Delete y Member tienen un tiempo de ejecución esperado O(1), siempre que la función hash distribuya adecuadamente y las colisiones se mantengan controladas.

Operaciones

- **Init():** inicializa la lista vacía.

- **Done():** libera los recursos utilizados por la lista.
- **Clear():** elimina todos los elementos de la lista.
- **Insert(elemento):** busca la posición correspondiente e inserta el elemento, si el elemento ya existe, la operación no tiene ningún efecto.
- **Delete(elemento):** elimina el elemento dentro de la lista, si no existe, la operación no tiene ningún efecto.
- **Member(elemento):** determina si el elemento pertenece a la lista.
- **Print():** recorre e imprime los elementos almacenados en la lista.
- **Union(A, B):** crea una nueva tabla hash que contiene los elementos de A y B sin duplicados.
- **Intersection(A, B):** crea una nueva tabla hash con los elementos que pertenecen a ambos conjuntos.
- **Difference(A, B):** crea una nueva tabla hash con los elementos que pertenecen a A pero no a B.

Tabla Hash Abierta

Una tabla hash es una estructura de datos que permite almacenar y recuperar elementos utilizando una función hash para calcular la posición en la que se encuentran. En esta implementación, las colisiones se resuelven mediante el método de encadenamiento separado.

Este método consiste en que cada posición de la tabla contiene una lista independiente. Cuando dos o más elementos generan el mismo índice mediante la función hash, se almacenan en la lista correspondiente a esa posición. De esta forma, una colisión no provoca que un elemento sustituya a otro.

Propiedades

- Crea una posición para cada elemento. En este caso, usa la función hash para generar una posición utilizando los caracteres del elemento y el módulo del tamaño de la tabla.
- Esta estructura resuelve las colisiones mediante encadenamiento, lo cual es que cuando dos elementos tienen la misma clave, se almacenan en una lista en esa clave.
- El tamaño de la tabla se establece al inicializarla.
- Las operaciones tienen un comportamiento esperado de O(1).
- Cada posición cuenta con una lista independiente. Si existen colisiones, las listas crecen y las pueden volverse O(n).
- Para insertar o eliminar un número no hay necesidad de desplazar los elementos en la tabla.

Operaciones

- **Init():** inicializa la lista vacía.
- **Done():** libera los recursos utilizados por la lista.
- **Clear():** se recorre cada posición en la tabla y por cada posición se vacía su contenido.
- **Insert(elemento):**calcula la posición del elemento utilizando la función hash, si el elemento no se encuentra en la posición de la tabla se inserta en esa posición.
- **Delete(elemento):** calcula la posición del elemento utilizando la función hash, si el elemento se encuentra en la posición de la tabla se elimina el elemento de esa posición.

- **Member(elemento):** calcula la posición del elemento utilizando la función hash, verifica que se el elemento se encuentre en esa posición, sí sí pertenece, retorna true, si no, retorna false.
- **Print():** Se recorre cada posición en la tabla a través de iteraciones en el rango del tamaño de la tabla, en cada posición se recorre la lista dentro de esa posición e imprime sus elementos, si no se encuentra algún elemento imprime “None”.
- **Union(A, B):** crea una tabla hash llamada “resultado” que va a contener los elementos de A y B. Recorre A en cada posición de la tabla, después recorre cada elemento en la lista de esa posición y lo inserta a resultado, luego recorre B en cada posición de la tabla, después recorre cada elemento en la lista de esa posición y lo inserta a resultado.
- **Intersection(A, B):** crea una tabla hash llamada “resultado” que va a contener los elementos de A y B. Recorre A en cada posición de la tabla y después recorre cada elemento en la lista de esa posición. Verifica con member si el elemento pertenece a B, si es así, lo inserta.
- **Difference(A, B):** crea una tabla hash llamada “resultado” que va a contener los elementos de A y B. Recorre A en cada posición de la tabla y después recorre cada elemento en la lista de esa posición. Verifica con member si el elemento no pertenece a B, si es así, lo inserta.

Función Hash

La función hash inicializa la variable suma en 0, esta variable irá acumulando el valor de hash mientras recorre carácter por carácter el elemento.

Recorremos elemento y actualizamos la variable suma con suma * 31 + ord(caracter), la función de ord() nos permite convertir el carácter a su valor Unicode para poder realizar las operaciones correspondientes. Se utiliza la constante 31 porque ayuda a que los caracteres y su posición en el elemento sean importantes para distribuir los resultados en distintos valores.

Al final de la función retorna suma % self.tamaño, esto devuelve (suma) mod (tamaño de la tabla hash), este retorno es el que genera el valor de la clave para el elemento que queremos insertar en la tabla hash.

El resultado es un índice comprendido entre 0 y tamaño - 1, que determina la lista en la que se almacenará el elemento.

La función es determinista: una misma cadena, procesada con el mismo tamaño de tabla, genera siempre el mismo índice. Sin embargo, cadenas diferentes pueden producir el mismo índice, lo que se conoce como una colisión

La distribución puede evaluarse comparando la cantidad de elementos de cada posición con la cantidad promedio esperada. Para una tabla de tamaño m y n elementos, dicha cantidad promedio es n/m.

Esta evaluación permite determinar empíricamente cómo se comporta la función hash para los datos seleccionados. No demuestra que la función sea verdaderamente aleatoria ni garantiza

que todas las cadenas posibles se distribuyan uniformemente, pero proporciona información sobre su comportamiento práctico.

Redistribución de su tiempo de duración

En la clase Conjunto Hash, la redistribución consiste en duplicar la capacidad de self.tamaño y crear un nuevo arreglo self.tabla para redistribuir los elementos cuando las celdas se saturan, manteniendo las búsquedas eficientes.

Flujo en el código:

1. Nuevo tamaño: Se define un nuevo tamaño (por ejemplo, nuevo_tamaño = self.tamaño * 2).

2. Re-hashing: Se recorre cada sublista de self.tabla y se recalcula el índice de cada elemento mediante self.hash(elemento) usando el nuevo valor de self.tamaño.

3. Reubicación: Se insertan los elementos en el nuevo arreglo y se actualizan los atributos self.tabla y self.tamaño del objeto.

Evaluación del tiempo de duración:

- Costo Esperado $O(N + M)$: Donde N es el total de elementos y M la cantidad de celdas (self.tamaño). Se recorren las M celdas e insertan los N elementos de forma directa (.append) en el nuevo arreglo.

- Impacto: Aunque una redistribución individual toma tiempo proporcional a N, al ejecutarse de forma esporádica permite que las operaciones insert, member y delete mantengan un costo promedio de $O(1)$.

Árbol de Búsqueda Binaria (ABB) genérico

Un Árbol de Búsqueda Binaria es una estructura de datos jerárquica formada por nodos. Cada nodo almacena un elemento y tiene, como máximo, un hijo izquierdo y un hijo derecho. Para un criterio de comparación consistente, los elementos menores que el nodo se ubican en el subárbol izquierdo y los mayores en el subárbol derecho. En este proyecto los elementos son cadenas alfanuméricas de hasta 20 caracteres, por lo que la comparación de cadenas determina su ubicación.

Propiedades

• Cada elemento se almacena una sola vez para respetar la propiedad de conjunto.
• La raíz identifica el inicio del árbol; un árbol vacío no tiene raíz.
• Una búsqueda compara el elemento buscado con el nodo actual y continúa por el subárbol izquierdo o derecho.
• Un recorrido inorden visita primero el subárbol izquierdo, después el nodo y finalmente el subárbol derecho;
por ello, produce los elementos en orden ascendente.
• El rendimiento depende de la forma del árbol: si está razonablemente equilibrado, las operaciones de búsqueda suelen ser más eficientes; si se desbalancea, pueden acercarse a un recorrido lineal.

Operaciones del Modelo Conjunto

• Init inicializa el árbol vacío; Done libera los recursos; Clear elimina los nodos; Insert incorpora un elemento respetando el orden y sin duplicarlo
• Delete elimina el elemento si existe
• Member busca su pertenencia
• Print muestra los elementos, preferiblemente mediante recorrido inorden; Union, Intersection y Difference generan un nuevo conjunto representado con la misma variante de ABB.

ABB por punteros

En la representación por punteros, cada nodo contiene el elemento y referencias al hijo izquierdo y al hijo derecho. La estructura conserva una referencia a la raíz. Los nodos pueden estar ubicados en distintas posiciones de memoria; las relaciones entre ellos se establecen mediante las referencias.

Propiedades y funcionamiento

• Un nodo puede tener cero, uno o dos hijos.
• La raíz es None cuando el árbol está vacío.
• Insert busca la posición de acuerdo con las comparaciones y crea un nodo nuevo si el elemento no existe.
• Delete debe contemplar los casos de nodo hoja, nodo con un hijo y nodo con dos hijos.
• Member sigue las comparaciones desde la raíz hasta encontrar el elemento o llegar a un enlace vacío.
• Print puede utilizar recorrido inorden para mostrar los elementos ordenados

A.B.B. por vector heap

En la representación por vector heap, cada nodo se almacena en un vector. La raíz se encuentra en la posición 0, y las posiciones de hijo izquierdo e hijo derecho de cada nodo se

calculan mediante fórmulas basadas en el índice del nodo actual. Las posiciones vacías se representan con **None**, lo que permite conservar la estructura del árbol.

Propiedades y funcionamiento

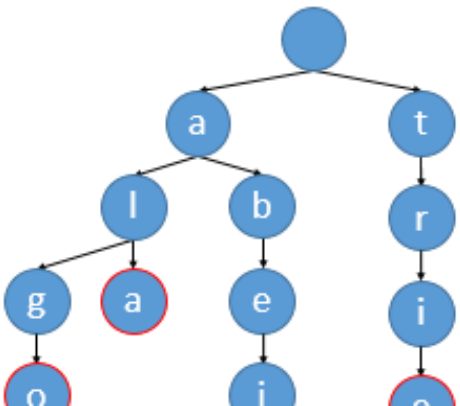
- La raíz se encuentra en el índice 0 del vector. Si el árbol está vacío, el vector no contiene elementos.
- El hijo izquierdo de un nodo ubicado en el índice i se encuentra en la posición 2 * i + 1, mientras que el hijo derecho se encuentra en la posición 2 * i + 2.
- La estructura conserva la propiedad del árbol binario de búsqueda: los elementos menores se ubican en el subárbol izquierdo y los mayores en el subárbol derecho.
- **Insert()** busca la posición correspondiente mediante las comparaciones y almacena el elemento en una posición vacía del vector. Si el elemento ya existe, la función no tiene efecto adicional.
- **Delete()** elimina el elemento sin alterar la propiedad de búsqueda del árbol. Debe contemplar los casos de nodo hoja, nodo con un hijo y nodo con dos hijos, conservando los índices de los demás nodos.
- **Member()** realiza la búsqueda desde la raíz, calculando el índice del hijo izquierdo o derecho según las comparaciones, hasta encontrar el elemento o llegar a una posición vacía.
- **Print()** puede utilizar un recorrido inorden para mostrar los elementos ordenados, visitando primero el subárbol izquierdo, después el nodo actual y finalmente el subárbol derecho.
- La estructura puede generar espacios vacíos entre los elementos del vector, representados mediante None, debido a la distribución de los nodos y según las fórmulas de los índices.
- La memoria utilizada depende de la posición más alta ocupada en el vector, por lo que un árbol muy desequilibrado puede generar espacios vacíos y aumentar el consumo de memoria.

E.D Genérica Trie

Descripción

Un Trie es una estructura de datos en forma de árbol que permite almacenar y buscar cadenas de caracteres de manera eficiente. Cada nodo representa un carácter o una parte de una cadena, y las palabras se forman siguiendo un camino desde la raíz hasta un nodo determinado. Cada nodo puede tener varios hijos, dependiendo de los caracteres posibles.

El Trie utiliza una marca especial para indicar cuándo termina una palabra, lo que permite almacenar palabras que comparten prefijos sin confundirlas entre sí. Por ejemplo, las palabras `casa`, `caso` y `cas` comparten parte de su recorrido dentro del árbol.



Esta estructura es especialmente útil para implementar diccionarios, autocompletado,

búsquedas por prefijos y correctores ortográficos. Su eficiencia depende principalmente de la longitud de las cadenas que se insertan o buscan, en lugar de depender directamente de la cantidad total de palabras almacenadas.

E.D. Trie por punteros

Descripción

- **Alfabeto:** 36 símbolos, `'0'..'9'` (índices 0 a 9) y `'a'..'z'` (índices 10 a 35).
- **Nodo:** un arreglo de 36 punteros a hijos (`None` es el puntero nulo) y una marca booleana `es_fin`.
- **Raíz:** representa la hilera vacía y no es un elemento.
- **Memoria dinámica:** los nodos se crean al insertar y se liberan al borrar o en Done.

Operaciones

- **Insert:** desciende por el trie creando los nodos que falten y marca el último como fin. Si el elemento ya existía, no cambia nada.
- **Member:** desciende por el trie; si se topa con un puntero nulo, el elemento no está, y si llega al final, devuelve la marca `es_fin`.
- **Delete:** desciende guardando el camino, quita la marca y luego libera de abajo hacia arriba los nodos que quedaron sin hijos y sin marca. Esta poda es necesaria para que la medición de memoria sea correcta.
- **Print:** recorre el trie en preorden con una pila explícita y no construye ninguna lista auxiliar.
- **Done / Clear:** Done desenlaza todos los nodos y anula la raíz, y Clear hace Done seguido de Init.
- **Union, Intersection y Difference:** devuelven un nuevo Trie por punteros, tal como exige el enunciado. Recorren el trie de A y consultan o insertan elemento a elemento.
    - **Union:** inserta en el resultado los elementos de A y luego los de B.
    - **Intersection:** de cada elemento de A, lo inserta si **`B.member(e)`**.
    - **Difference:** de cada elemento de A, lo inserta si no **`B.member(e).`**

E.D. Trie por arreglos

En la representación por arreglos, cada nodo contiene un arreglo de 36 posiciones (26 letras 'a'..'z' y 10 dígitos '0'..'9'), donde cada posición es una referencia al hijo correspondiente a ese carácter, y una bandera que indica si en ese nodo termina un elemento. Los elementos no se guardan completos en ningún nodo: se forman con la secuencia de caracteres del camino desde la raíz. La estructura conserva una referencia a la raíz, y los nodos pueden estar ubicados en distintas posiciones de memoria; las relaciones entre ellos se establecen mediante las posiciones del arreglo.

Propiedades y funcionamiento

- Un nodo puede tener entre cero y 36 hijos. La posición de cada hijo se calcula directamente a partir del carácter: 'a' - 'z' corresponden a las posiciones 0-25 y '0' - '9' a las posiciones 26-35.
- La raíz es un nodo sin hijos cuando el conjunto está vacío.
- Los elementos que comparten un prefijo comparten también los nodos de ese prefijo, y la unicidad de los elementos queda garantizada porque cada arreglo corresponde a un único camino.
- Un nodo con la bandera de fin activada indica que el camino hasta él forma un elemento del conjunto. Esta bandera permite distinguir un elemento de un simple prefijo de otro (por ejemplo, cas y casa).
- **Insert()** recorre el arreglo carácter por carácter, crea los nodos que falten en el camino y activa la bandera de fin en el último. Si el elemento ya existe, la operación no tiene efecto adicional.
- **Delete()** desactiva la bandera de fin del último nodo del camino y luego elimina, desde ese nodo hacia la raíz, los nodos que hayan quedado sin hijos y sin marcar fin de elemento. Los nodos compartidos con otros elementos se conservan.
- **Member()** sigue el camino del arreglo desde la raíz; el elemento pertenece al conjunto sólo si el camino existe completo y el último nodo tiene la bandera de fin activada.
- **Print()** utiliza un recorrido en profundidad que recorre los hijos en orden de posición, acumulando el prefijo; así los elementos se muestran ordenados (primero letras y luego dígitos).
- **Union**, **Intersection** y **Difference** recorren los dos tries en paralelo, posición por posición, y construyen un nuevo trie con el resultado. La unión copia las ramas que existen en un solo conjunto, la intersección conserva únicamente las ramas presentes en ambos, y la diferencia conserva las ramas de A descartando los elementos que también están en B.
- El costo de **Insert**, **Member** y **Delete** depende de la longitud del elemento (máximo 20) y no de la cantidad de elementos almacenados.